

Εργαστήριο Βάσεων Δεδομένων

Project Εξαμήνου



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

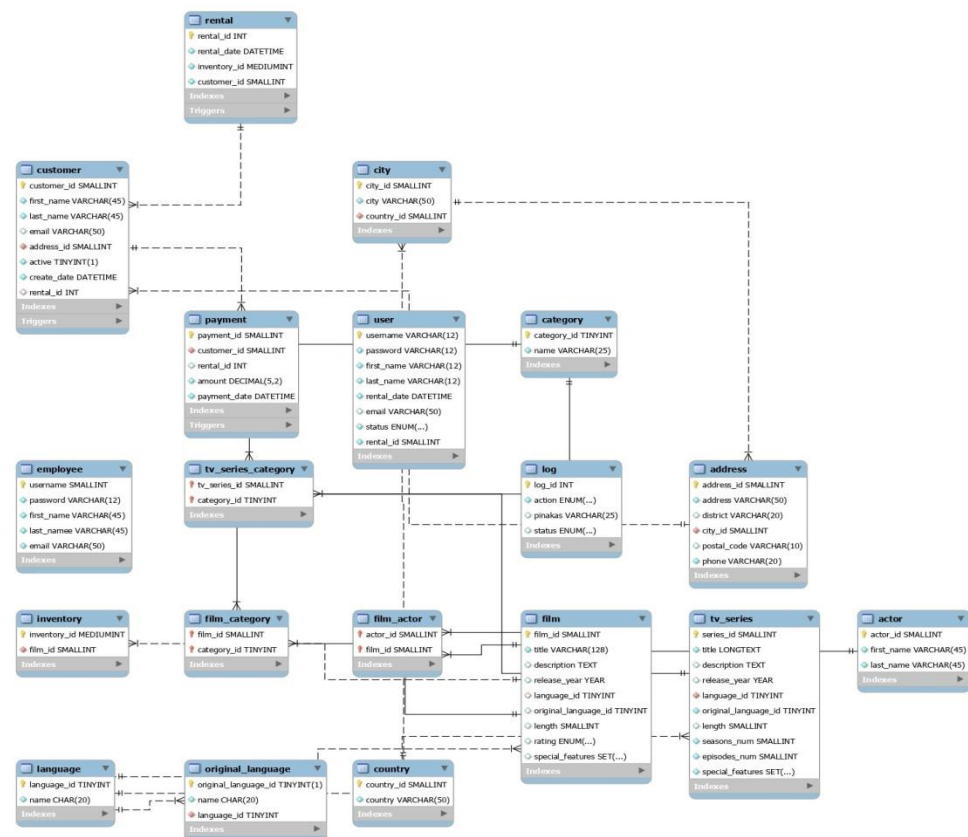


Εργασία Φοιτητών : Χρήστος Θανάτη AM:1058106 , Έτος:6ο

Ευαγγελία Ρίνη AM:1057777 , Έτος:6ο

Μέρος Α

1) A) EER-DIAGRAM



B) Για τις ανάγκες του πρώτου μέρους της εργαστηριακής άσκησης δημιουργήσαμε τους ακόλουθους πίνακες:

- Την διατήρηση πίνακα ενεργειών (log), ο πίνακας ενημερώνεται κάθε φορά που επιχειρείται μια ενέργεια εισαγωγής, επεξεργασίας ή διαγραφής δεδομένων και θα καταγράφει το username του χρήστη, την ημερομηνία και ώρα του συμβάντος, το αν εκτελέστηκε επιτυχώς ή όχι, το είδος της (insert, update, ή delete) και το όνομα του πίνακα που αφορούσε.
- Σειρές (tv_series), όπου για κάθε σειρά αποθηκεύονται ένας μοναδικός κωδικός, ο τίτλος, περιγραφή της ταινίας, το έτος που κυκλοφόρησε, η γλώσσα ή οι γλώσσες στις οποίες διατίθεται, η διάρκειά της, η σήμανση καταλληλότητας και ειδικά χαρακτηριστικά της έκδοσης.
- Κατηγορία Σειρών (tv_series_category), όπου για κάθε σειρά περιέχει το κωδικό όνομα κάθε σειράς.

- Υπάλληλοι του παρόχου (employees), που ενημερώνουν το διαθέσιμο τηλεοπτικό περιεχόμενο της υπηρεσίας και παρακολουθούν την λειτουργία της υπηρεσίας.
- Χρήστης(user) ,όπου μέσω του γνωρίσματος status ξεχωρίζουμε την ιδιότητα του (διαχειριστής – υπάλληλος- πελάτης).
- Αρχική γλώσσα (original language), όπου περιέχει τα γνωρίσματα του πίνακα language με επιπλέον γνώρισμα ένα κωδικό που χρειαζόμαστε για την καλύτερη εξυπηρέτηση των πελατών ώστε να υπάρχει αυτόματη προεπιλογή γλώσσας.

2) •CREATE TABLE

```
CREATE TABLE `log` (
  `log_id` int NOT NULL AUTO_INCREMENT,
  `action` enum('insert','update','delete') NOT NULL,
  `pinakas` varchar(25) DEFAULT NULL,
  `status` enum('administrator','employee','costumer') DEFAULT 'costumer',
  PRIMARY KEY (`log_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
CREATE TABLE `tv_series` (
  `series_id` smallint unsigned NOT NULL AUTO_INCREMENT,
  `title` longtext NOT NULL,
  `description` text,
  `release_year` year DEFAULT NULL,
  `language_id` tinyint unsigned NOT NULL,
  `original_language_id` tinyint unsigned NOT NULL,
  `length` smallint unsigned DEFAULT NULL,
  `seasons_num` smallint NOT NULL,
  `episodes_num` smallint NOT NULL,
```

```
`special_features` set('Trailers','Commentaries','Deleted Scenes','Behind the Scenes')
NOT NULL,

PRIMARY KEY (`series_id`),

KEY `fk_series_language` (`language_id`),

CONSTRAINT `fk_series_language` FOREIGN KEY (`language_id`) REFERENCES
`language` (`language_id`)

) ENGINE=InnoDB AUTO_INCREMENT=828 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci
```

```
CREATE TABLE `tv_series_category` (

`tv_series_id` smallint unsigned NOT NULL,

`category_id` tinyint unsigned NOT NULL,

PRIMARY KEY (`tv_series_id`,`category_id`),

KEY `category_id` (`category_id`),

CONSTRAINT `category_id` FOREIGN KEY (`category_id`) REFERENCES `category`
(`category_id`) ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT `tv_series_id` FOREIGN KEY (`tv_series_id`) REFERENCES `tv_series`
(`series_id`) ON DELETE RESTRICT ON UPDATE CASCADE

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
CREATE TABLE `employee` (

`username` smallint unsigned NOT NULL AUTO_INCREMENT,

`password` varchar(12) NOT NULL,

`first_name` varchar(45) NOT NULL,

`last_name` varchar(45) NOT NULL,

`email` varchar(50) NOT NULL,

PRIMARY KEY (`username`)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
CREATE TABLE `user` (  
  `username` varchar(12) NOT NULL,  
  `password` varchar(12) NOT NULL,  
  `first_name` varchar(12) NOT NULL,  
  `last_name` varchar(12) NOT NULL,  
  `rental_date` datetime NOT NULL,  
  `email` varchar(50) DEFAULT NULL,  
  `status` enum('administrator','employee','costumer') NOT NULL,  
  `rental_id` smallint NOT NULL,  
  PRIMARY KEY (`username`),  
  UNIQUE KEY `rental_date_UNIQUE` (`rental_date`),  
  KEY `fk_user_rental_idx` (`rental_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
CREATE TABLE `original_language` (  
  `original_language_id` tinyint(1) NOT NULL,  
  `name` char(20) NOT NULL,  
  `language_id` tinyint unsigned NOT NULL AUTO_INCREMENT,  
  PRIMARY KEY (`original_language_id`),  
  KEY `language_id_idx` (`language_id`),  
  CONSTRAINT `language_id` FOREIGN KEY (`language_id`) REFERENCES `language`  
  (`language_id`) ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

•INSERTS

```
INSERT INTO `tv_series` (`series_id`, `title`, `description`, `release_year`, `language_id`,  
`original_language_id`, `length`, `seasons_num`, `episodes_num`, `special_features`)
```

```
VALUES(820, 'Game Of Thrones', 'In the mythical continent of Westeros,nine families of  
higher nobility scramble bitterly to gain power over the seven kingdoms and the Iron  
throne.', 2011, 1, 1, 124, 12,10, 'Behind the Scenes'),
```

```
(612, 'Breaking Bad', 'A modest high school chemistry teacher, who discovers a new  
purpose in life when he learns he has terminal cancer and turns to a life of crime to provide  
for his family', 2008, 1, 1, 124, 5,15, 'Behind the Scenes');
```

```
INSERT INTO user
```

```
(username,password,first_name,last_name,rental_date,email,status,rental_id) VALUES
```

```
('NICKO', '234', 'NICKOLAS', 'MISTEL', '2008-06-15
```

```
01:25:08', 'NICKOLAS.MISTEL@sakilacustomer.org', 'customer', '152'),
```

```
('CHRISTO', '234', 'CHRIS', 'BROTHERS', '2005-05-26
```

```
08:44:08', 'CHRIS.BROTHERS@sakilacustomer.org', 'customer', '214');
```

```
INSERT INTO tv_series_category
```

```
(tv_series_id,category_id) VALUES ('820','15'),
```

```
('612','13');
```

3) •Store Procedures

3.1 DELIMITER \$\$

```
CREATE PROCEDURE `Proc1` ()
```

```
BEGIN
```

```
DROP PROCEDURE IF EXISTS Proc1;
```

```
CREATE PROCEDURE Proc1
```

```
(
```

```
    IN charactiras char(1),
```

```
    IN arithmos INT,
```

```
    IN begin_date DATE,
```

```
    IN end_date DATE
```

```
)
```

```

BEGIN

    IF (character LIKE 'm')

        THEN

            -- metraw plithos tainiwn kai metaonomazw

            SELECT COUNT(*) AS PLITHOS1,

            film.film_id AS TAINIA,

            film.title AS TITLOS1

            FROM rental

                INNER JOIN inventory ON rental.inventory_id =
inventory.inventory_id

                INNER JOIN film ON inventory.film_id = film.film_id

                WHERE rental.rental_date BETWEEN begin_date AND
end_date

                GROUP BY film.title

                ORDER BY Number DESC

                LIMIT 0, arithmos;

            -- metraw plithos seirwn kai metaonomazw

        ELSE SELECT COUNT(*) AS PLITHOS2,

            series.series_id AS SEIRA,

            series.title AS TITLOS2

            FROM rental

                INNER JOIN inventory ON rental.inventory_id =
inventory.inventory_id

                INNER JOIN series ON inventory.series_id = series.series_id

                WHERE rental.rental_date BETWEEN begin_date AND end_date

                GROUP BY series.title

                ORDER BY Number DESC

                LIMIT 0, arithmos;

    END IF;

```

```
END%%
```

```
DELIMITER ;
```

```
END
```

3.2

```
DROP PROCEDURE IF EXISTS Proc2;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE Proc2
```

```
(
```

```
    IN email VARCHAR(50),
```

```
    IN Hmerominia DATE
```

```
)
```

```
BEGIN
```

```
    SELECT COUNT (rental_id) AS Enoikiaseis
```

```
    FROM rental
```

```
    INNER JOIN customer ON rental.customer_id = customer.customer_id
```

```
    -- Koitaei to customer.email na einai idio me to email pou dinoume
```

```
    WHERE customer.email = email
```

```
    -- Koitaei to rental_date na einai idio me tin hmerominia pou dinoume
```

```
    and DATE(rental.rental_date) LIKE Hmerominia;
```

```
END$$
```

```
DELIMITER ;
```

3.4

```
a) DROP PROCEDURE IF EXISTS Proc4a;
```

```
DELIMITER %%
```

```
CREATE PROCEDURE Proc4a
```

```
(
    IN first_last_name VARCHAR(45),
    IN end_last_name VARCHAR(45)
)
BEGIN
    SELECT count(`actor_id`) FROM actor
        WHERE last_name between first_last_name and end_last_name;

    SELECT `first_name`, `last_name` FROM actor
        WHERE last_name between concat(first_last_name, '%') and concat( end_last_name,
        '%')
        ORDER BY last_name ASC;
END%%
DELIMITER ;
```

β)

```
DROP PROCEDURE IF EXISTS Proc4b;
DELIMITER %%
CREATE PROCEDURE Proc4b
(
    IN last_name VARCHAR(45)
)
BEGIN
    declare count_actors int;
    -- check if there are more than 1 results
    set count_actors = (select count(*) from actor where actor.last_name = last_name);
    IF (count_actors > 1) THEN
        select count_actors as plhthos;
```

```
END IF;  
  
select `actor_id`, `first_name`, `last_name` from actor  
  
where actor.last_name = last_name;  
  
END%%  
  
DELIMITER ;
```

•Triggers

4.1) Trigger που θα ενημερώνουν το σχετικό πίνακα καταγραφής ενεργειών (log) για κάθε ενέργεια εισαγωγής, ενημέρωσης ή διαγραφής στους πίνακες rental, payment , inventory.

→rental

```
CREATE DEFINER=`root`@`localhost` TRIGGER `rental_BEFORE_INSERT` BEFORE INSERT ON  
`rental` FOR EACH ROW BEGIN  
  
INSERT INTO log(action, pinakas) VALUES('insert', 'rental');  
  
END
```

```
CREATE DEFINER=`root`@`localhost` TRIGGER `rental_BEFORE_UPDATE` BEFORE UPDATE ON  
`rental` FOR EACH ROW BEGIN  
  
CALL ProcedureEkptwsi;  
  
END
```

→payment

```
CREATE DEFINER=`root`@`localhost` TRIGGER `payment_BEFORE_INSERT` BEFORE INSERT ON  
`payment` FOR EACH ROW BEGIN  
  
INSERT INTO log (action, pinakas) VALUES('insert', 'payment');  
  
END
```

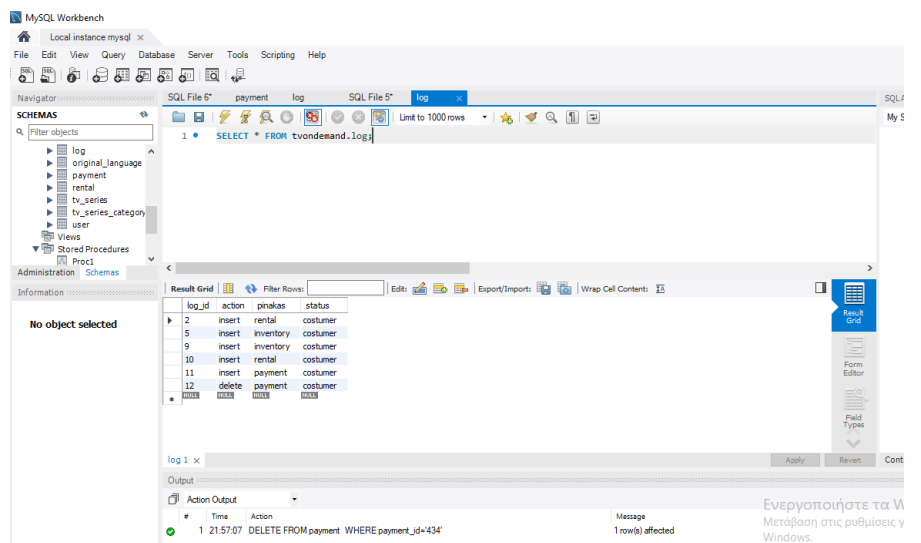
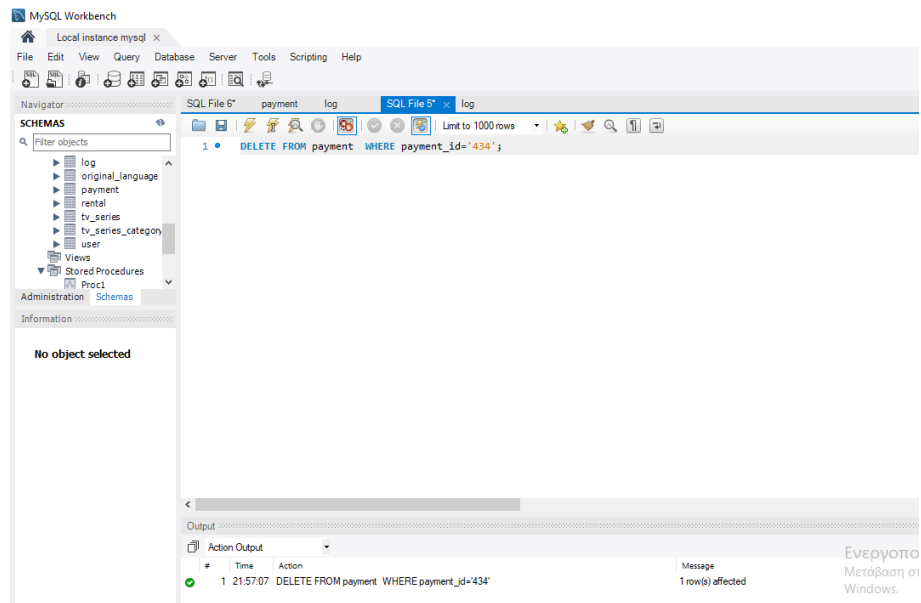
```
CREATE DEFINER=`root`@`localhost` TRIGGER `payment_BEFORE_UPDATE` BEFORE UPDATE  
ON `payment` FOR EACH ROW BEGIN  
  
INSERT INTO log (action, pinakas ) VALUES('update', 'payment');
```

END

```
CREATE DEFINER=`root`@`localhost` TRIGGER `payment_BEFORE_DELETE` BEFORE DELETE
ON `payment` FOR EACH ROW BEGIN

INSERT INTO log (action, pinakas ,) VALUES('delete', 'payment');

END
```



Διαγραφή από το table payment και ενημέρωση της διαγραφής στο table log.

→inventory

```
CREATE DEFINER='root'@'localhost' TRIGGER `inventory_BEFORE_INSERT` BEFORE INSERT
ON `inventory` FOR EACH ROW BEGIN

INSERT INTO log (action, pinakas ) VALUES('insert', 'inventory');

END
```

```
CREATE DEFINER='root'@'localhost' TRIGGER `inventory_BEFORE_UPDATE` BEFORE UPDATE
ON `inventory` FOR EACH ROW BEGIN

INSERT INTO log (action, pinakas ) VALUES('update', 'inventory');

END
```

```
CREATE DEFINER='root'@'localhost' TRIGGER `inventory_BEFORE_DELETE` BEFORE DELETE
ON `inventory` FOR EACH ROW BEGIN

INSERT INTO log (action, pinakas ) VALUES('delete', 'inventory');

END
```

```
4.2) DELIMITER $$

CREATE TRIGGER EKPTWSH

BEFORE UPDATE ON rental FOR EACH ROW

BEGIN

    CALL ProcedureEkptwsi;

END;

END#

DELIMITER ;
```

Το συγκεκριμένο trigger πυροδοτείτε όταν καλέσουμε την παρακάτω Store Procedure η οποία δεν είναι υλοποιήσιμη από το πρόγραμμα, παρόλο αυτά η σύνταξη της είναι η ακόλουθη.

```
DROP PROCEDURE IF EXISTS ProcedureEkptwsi;

DELIMITER $$

CREATE PROCEDURE ProcedureEkptwsi
(
    IN Username VARCHAR(25),
    IN Hmerominia DATE
)
BEGIN
    SELECT COUNT (rental_id) AS Enoikiaseis
    FROM rental
    INNER JOIN customer ON rental.customer_id=customer.customer_id
    WHERE rental.rental_date=CURRENT_TIMESTAMP
    GROUP BY customer.email;

    IF Enoikiaseis>=3 THEN
        SELECT amount FROM rental INNER JOIN payment
        ON rental.rental_id=payment.rental_id;

        UPDATE payment.amount SET payment.amount=payment.default_price/2;
    ELSE
        SELECT updated_amount FROM rental INNER JOIN payment
        ON rental.rental_id=payment.rental_id;

        UPDATE payment.amount SET payment.amount=payment.default_price;
    END IF;
END;

END$$
```

4.3) DELIMITER \$\$

```
CREATE TRIGGER DENIED
```

BEFORE UPDATE

ON customer FOR EACH ROW

BEGIN

IF (old.`customer\_id` <> new.`customer\_id`) THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'customer_id cannot change';

ELSEIF (old.`first\_name` <> new.`first\_name`) THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A customers first name cant change';

ELSEIF (old.`last\_name` <> new.`last\_name`) THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A customers last name cant change';

ELSEIF (old.`create\_date` <> new.`create\_date`) THEN

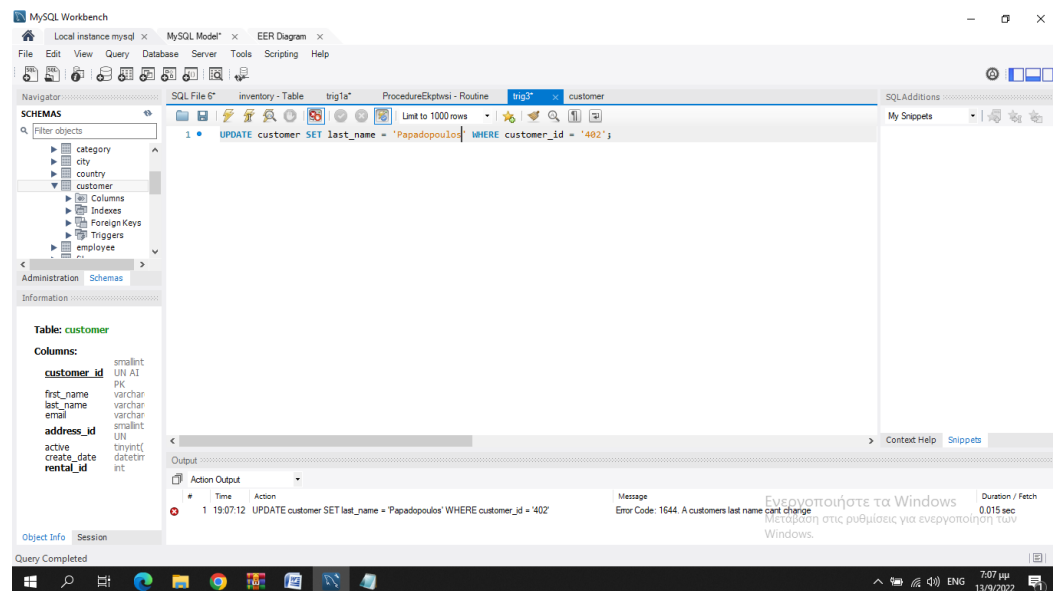
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'A customers created date cant change';

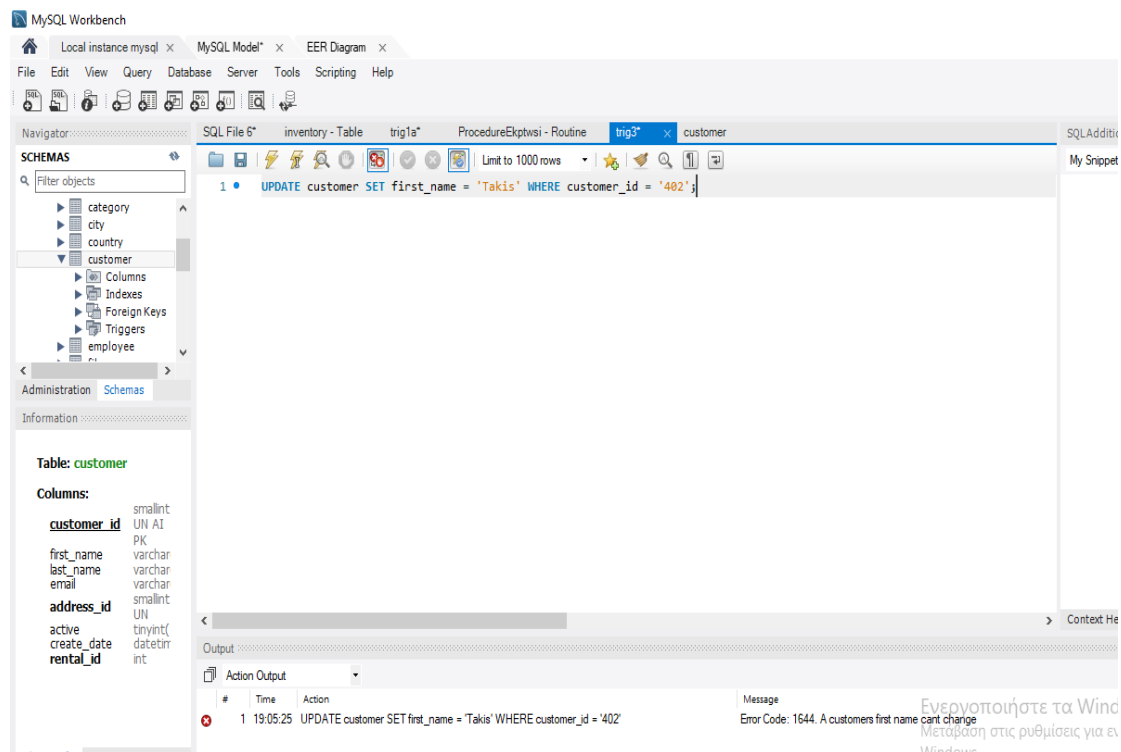
END IF;

END\$\$

DELIMITER ;

Screenshot έπαιτα από την πυροδότηση του:





Το trigger πυροδοτήθηκε διότι έγινε προσπάθεια να αλλάξει ονοματεπώνυμο ο customer.

Παραδοχές: Προσπαθήσαμε να υλοποιήσουμε την Store Procedure του ερωτήματος 3.3 αλλά δεν καταφέραμε να την τρέξουμε. Ωστόσο το σκεπτικό μας για την υλοποίηση της είναι το ακόλουθο:

```
DROP PROCEDURE IF EXISTS Proc3;

delimiter %%

create procedure Proc3

BEGIN

    -- counter = metraei tous mhnes apo ton 1o ews ton 12o

    declare counter int;

    declare film_count_both int;

    declare film_count int;

    declare series_count_both int;

    declare series_count int;
```

```

declare earnings float;

set counter = 0;

while counter <= 12 do

    -- film_count = to noumero tw n atomwn pou exoun enikiasei tainies, kai
    plhrwnoun 0.4 € ana tainia

    set film_count = (

        SELECT count(film.film_id) as Films FROM film

        INNER JOIN inventory ON film.film_id=inventory.film_id

        INNER JOIN rental ON inventory.inventory_id=rental.inventory_id

        WHERE rental.rental_date LIKE counter

    );

    -- series_count = to noumero tw n atomwn pou exoun enikiasei seires, kai plhrwnoun 0.2
    € ana epeisodio seiras

    set series_count = (

        SELECT count(series_id) from series

        WHERE series.series_id IN (

            SELECT series.series_id AS Series FROM series

            INNER JOIN inventory ON series.series_id =
inventory.series_id

            INNER JOIN rental ON inventory.inventory_id =
rental.inventory_id

            WHERE rental.rental_date LIKE counter

        );

        -- film_count_both = to noumero tw n atomwn pou exoun enikiasei tainies,
        kai plhrwnoun 0.3 € ana tainia

        set film_count_both = (

            SELECT count(film.film_id) as Number_of_films FROM film

```

```

        INNER JOIN inventory ON film.film_id=inventory.film_id

        INNER JOIN rental ON inventory.inventory_id=rental.inventory_id

        WHERE month(rental.rental_date) LIKE counter

    );

    -- series_count_both = to noumero tw n atomwn pou exoun enikiasei seires,
    kai plhrwnoun 0.1 € ana epeisodio seiras

    set series_count_both = (

        SELECT count(series.series_id) AS Number_of_series FROM series

        INNER JOIN inventory ON series.series_id =
inventory.series_id

        INNER JOIN rental ON inventory.inventory_id =
rental.inventory_id

        WHERE month(rental.rental_date) LIKE counter

    );

    set earnings = film_count_both * 0.3 + film_count * 0.4 + series_count * 0.1
+ series_count_both * 0.2;

    -- emfanizw ta apotelesmata se mhnes

    IF (counter = 0) THEN

        select earnings as January_Earnings;

    ELSE IF (counter = 1) THEN

        select earnings as February_Earnings;

    ELSE IF (counter = 2) THEN

        select earnings as March_Earnings;

    ELSE IF (counter = 3) THEN

        select earnings as April_Earnings;

    ELSE IF (counter = 4) THEN

        select earnings as May_Earnings;

    ELSE IF (counter = 5) THEN

        select earnings as June_Earnings;

```

```
ELSE IF (counter = 6) THEN
    select earnings as July_Earnings;
ELSE IF (counter = 7) THEN
    select earnings as August_Earnings;
ELSE IF (counter = 8) THEN
    select earnings as September_Earnings;
ELSE IF (counter = 9) THEN
    select earnings as October_Earnings;
ELSE IF (counter = 10) THEN
    select earnings as November_Earnings;
ELSE IF (counter = 11) THEN
    select earnings as December_Earnings;

END IF;

set counter = counter +1;

end while;

end$$

DELIMITER ;
```

Υλοποιήθηκε μόνο το Α' Μέρος της εργασίας.